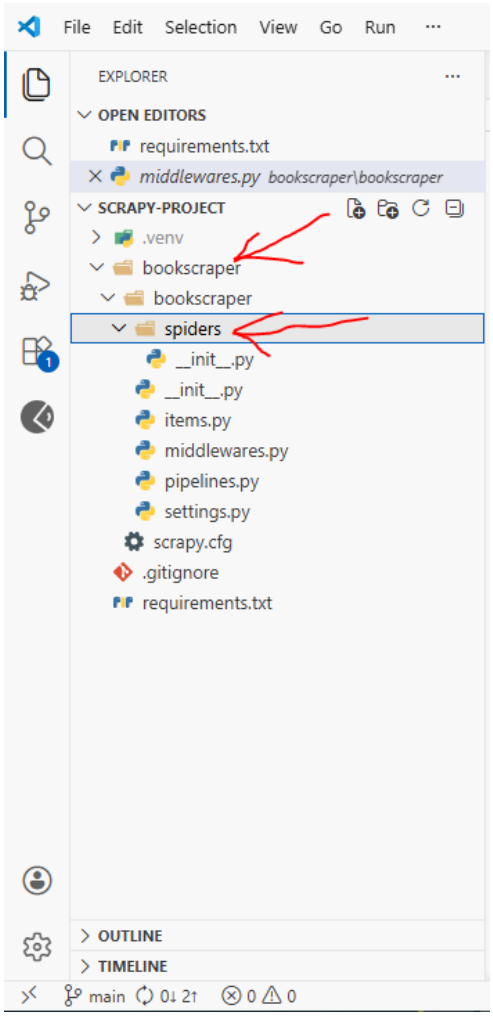
To create a new Scrapy project: **scrapy startproject project-name-here**

****

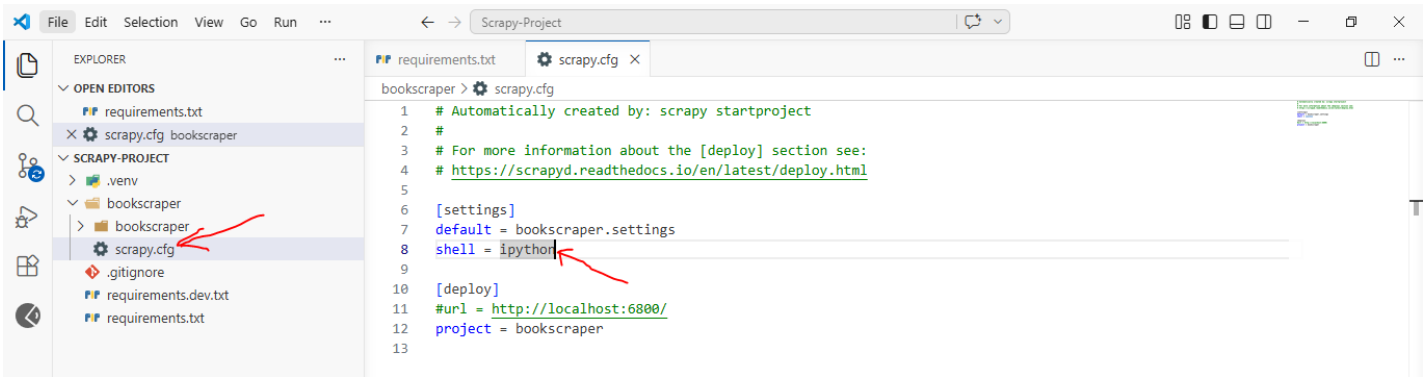


****

```
(.venv) D:\j-1-qa-testing\Scrapy-Project\bookscraper\bookscraper\spiders>scrapy genspider bookspider books,toscrape.com
Created spider 'bookspider' using template 'basic' in module:
  bookscraper.spiders.bookspider

(.venv) D:\j-1-qa-testing\Scrapy-Project\bookscraper\bookscraper\spiders>
```

****



****

```
(.venv) D:\j-l-qa-testing\Scrapy-Project>scrapy shell
2026-05-02 21:11:26 [scrapy.utils.log] INFO: Scrapy 2.15.2 started (bot: scrapybot)
2026-05-02 21:11:27 [scrapy.utils.log] INFO: Versions:
{'lxml': '6.1.0',
 'libxml2': '2.11.9',
 'cssselect': '1.4.0',
 'parsel': '1.11.0',
 'w3lib': '2.4.1',
 'Twisted': '25.5.0',
 'Python': '3.12.10 (tags/v3.12.10:0cc8128, Apr  8 2025, 12:21:36) [MSC v.1943 '
           '64 bit (AMD64)]',
 'pyOpenSSL': '26.1.0 (OpenSSL 4.0.0 14 Apr 2026)',
 'cryptography': '47.0.0',
 'Platform': 'Windows-10-10.0.19045-SP0'}
2026-05-02 21:11:27 [scrapy.crawler] DEBUG: Using AsyncCrawlerProcess
2026-05-02 21:11:27 [asyncio] DEBUG: Using selector: SelectSelector
2026-05-02 21:11:27 [scrapy.addons] INFO: Enabled addons:
[]
2026-05-02 21:11:28 [scrapy.utils.log] DEBUG: Using reactor: twisted.internet.asyncioreactor.AsyncioSelectorReactor
2026-05-02 21:11:28 [scrapy.utils.log] DEBUG: Using asyncio event loop: asyncio.windows_events._WindowsSelectorEventLoop
2026-05-02 21:11:28 [scrapy.extensions.telnet] INFO: Telnet Password: 9a52e415f926b849
2026-05-02 21:11:28 [scrapy.middleware] INFO: Enabled extensions:
['scrapy.extensions.corestats.CoreStats',
 'scrapy.extensions.logcount.LogCount',
 'scrapy.extensions.telnet.TelnetConsole']
2026-05-02 21:11:28 [scrapy.crawler] INFO: Overridden settings:
{'DUPEFILTER_CLASS': 'scrapy.dupefilters.BaseDupeFilter',
 'LOGSTATS_INTERVAL': 0}
2026-05-02 21:11:30 [scrapy.middleware] INFO: Enabled downloader middlewares:
['scrapy.downloadermiddlewares.offsite.OffsiteMiddleware',
 'scrapy.downloadermiddlewares.httpauth.HttpAuthMiddleware',
 'scrapy.downloadermiddlewares.downloadtimeout.DownloadTimeoutMiddleware',
 'scrapy.downloadermiddlewares.defaultheaders.DefaultHeadersMiddleware',
 'scrapy.downloadermiddlewares.useragent.UserAgentMiddleware',
 'scrapy.downloadermiddlewares.retry.RetryMiddleware',
 'scrapy.downloadermiddlewares.redirect.MetaRefreshMiddleware',
 'scrapy.downloadermiddlewares.httpcompression.HttpCompressionMiddleware',
 'scrapy.downloadermiddlewares.redirect.RedirectMiddleware',
 'scrapy.downloadermiddlewares.cookies.CookiesMiddleware',
 'scrapy.downloadermiddlewares.httpproxy.HttpProxyMiddleware',
 'scrapy.downloadermiddlewares.stats.DownloaderStats']
2026-05-02 21:11:30 [scrapy.middleware] INFO: Enabled spider middlewares:
['scrapy.spidermiddlewares.start.StartSpiderMiddleware',
```

****

```
2026-05-02 21:11:30 [scrapy.middleware] INFO: Enabled item pipelines:
[]
2026-05-02 21:11:30 [scrapy.extensions.telnet] INFO: Telnet console listening on 127.0.0.1:6023
[s] Available Scrapy objects:
[s]   scrapy     scrapy module (contains scrapy.Request, scrapy.Selector, etc)
[s]   crawler    <scrapy.crawler.Crawler object at 0x000002C0EFB5C8C0>
[s]   item       {}
[s]   settings   <scrapy.settings.Settings object at 0x000002C0EFDA7500>
[s] Useful shortcuts:
[s]   fetch(url[, redirect=True]) Fetch URL and update local objects (by default, redirects are followed)
[s]   fetch(req)                  Fetch a scrapy.Request and update local objects
[s]   shelp()           Shell help (print this help)
[s]   view(response)    View response in a browser
Tip: Use `ipython --help-all | less` to view all the IPython configuration options.

2026-05-02 21:11:33 [asyncio] DEBUG: Using selector: SelectSelector
In [1]: result = fetch('https://books.toscrape.com')
2026-05-02 21:12:04 [scrapy.core.engine] INFO: Spider opened
2026-05-02 21:12:06 [scrapy.core.engine] DEBUG: Crawled (200) <GET https://books.toscrape.com> (referer: None)

2026-05-02 21:12:06 [asyncio] DEBUG: Using selector: SelectSelector
```

****

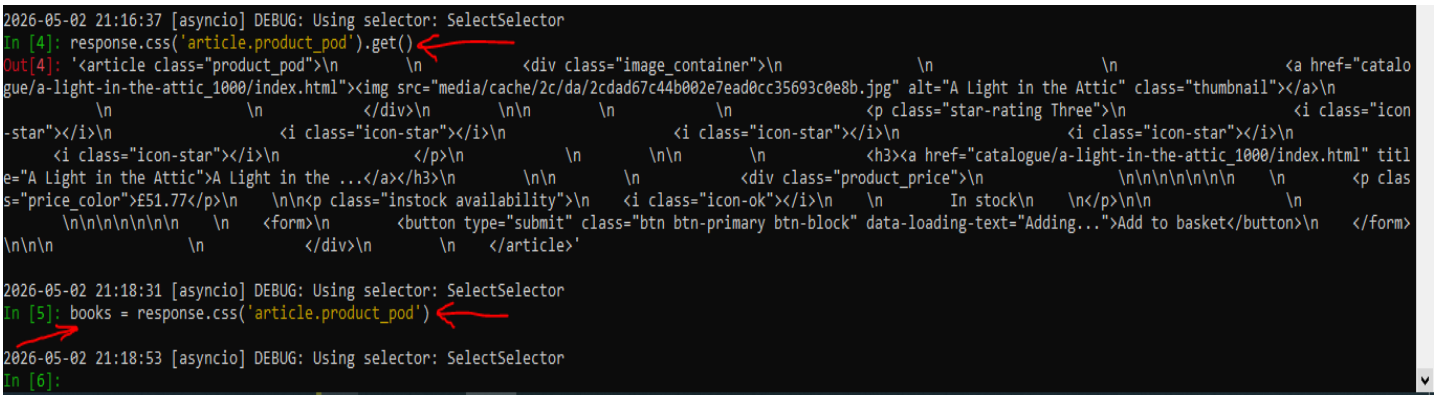
```
In [2]: response
Out[2]: <200 https://books.toscrape.com>

2026-05-02 21:14:54 [asyncio] DEBUG: Using selector: SelectSelector
In [3]:
```

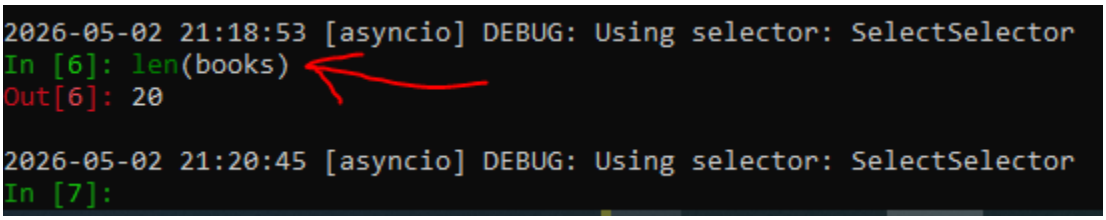
****

```
2026-05-02 21:14:54 [asyncio] DEBUG: Using selector: SelectSelector
In [3]: response.css('article.product_pod')
Out[3]:
[<Selector query="descendant-or-self::article[@class and contains(concat(' ', normalize-space(@class), ' '), ' product_pod ')]" data='<article class="product_pod">\n
    ...'>,
 <Selector query="descendant-or-self::article[@class and contains(concat(' ', normalize-space(@class), ' '), ' product_pod ')]" data='<article class="product_pod">\n
    ...'>,
 <Selector query="descendant-or-self::article[@class and contains(concat(' ', normalize-space(@class), ' '), ' product_pod ')]" data='<article class="product_pod">\n
    ...'>,
 <Selector query="descendant-or-self::article[@class and contains(concat(' ', normalize-space(@class), ' '), ' product_pod ')]" data='<article class="product_pod">\n
    ...'>,
 <Selector query="descendant-or-self::article[@class and contains(concat(' ', normalize-space(@class), ' '), ' product_pod ')]" data='<article class="product_pod">\n
    ...'>,
 <Selector query="descendant-or-self::article[@class and contains(concat(' ', normalize-space(@class), ' '), ' product_pod ')]" data='<article class="product_pod">\n
    ...'>,
 <Selector query="descendant-or-self::article[@class and contains(concat(' ', normalize-space(@class), ' '), ' product_pod ')]" data='<article class="product_pod">\n
    ...'>,
 <Selector query="descendant-or-self::article[@class and contains(concat(' ', normalize-space(@class), ' '), ' product_pod ')]" data='<article class="product_pod">\n
    ...'>,
 <Selector query="descendant-or-self::article[@class and contains(concat(' ', normalize-space(@class), ' '), ' product_pod ')]" data='<article class="product_pod">\n
    ...'>,
 <Selector query="descendant-or-self::article[@class and contains(concat(' ', normalize-space(@class), ' '), ' product_pod ')]" data='<article class="product_pod">\n
    ...'>,
 <Selector query="descendant-or-self::article[@class and contains(concat(' ', normalize-space(@class), ' '), ' product_pod ')]" data='<article class="product_pod">\n
    ...'>,
 <Selector query="descendant-or-self::article[@class and contains(concat(' ', normalize-space(@class), ' '), ' product_pod ')]" data='<article class="product_pod">\n
    ...'>,
 <Selector query="descendant-or-self::article[@class and contains(concat(' ', normalize-space(@class), ' '), ' product_pod ')]" data='<article class="product_pod">\n
    ...'>,
 <Selector query="descendant-or-self::article[@class and contains(concat(' ', normalize-space(@class), ' '), ' product_pod ')]" data='<article class="product_pod">\n
    ...'>,
 <Selector query="descendant-or-self::article[@class and contains(concat(' ', normalize-space(@class), ' '), ' product_pod ')]" data='<article class="product_pod">\n
    ...'>,
 <Selector query="descendant-or-self::article[@class and contains(concat(' ', normalize-space(@class), ' '), ' product_pod ')]" data='<article class="product_pod">\n
    ...'>,
 <Selector query="descendant-or-self::article[@class and contains(concat(' ', normalize-space(@class), ' '), ' product_pod ')]" data='<article class="product_pod">\n
    ...'>,
 <Selector query="descendant-or-self::article[@class and contains(concat(' ', normalize-space(@class), ' '), ' product_pod ')]" data='<article class="product_pod">\n
    ...'>,
 <Selector query="descendant-or-self::article[@class and contains(concat(' ', normalize-space(@class), ' '), ' product_pod ')]" data='<article class="product_pod">\n
    ...'>,
 <Selector query="descendant-or-self::article[@class and contains(concat(' ', normalize-space(@class), ' '), ' product_pod ')]" data='<article class="product_pod">\n
    ...'>]
```

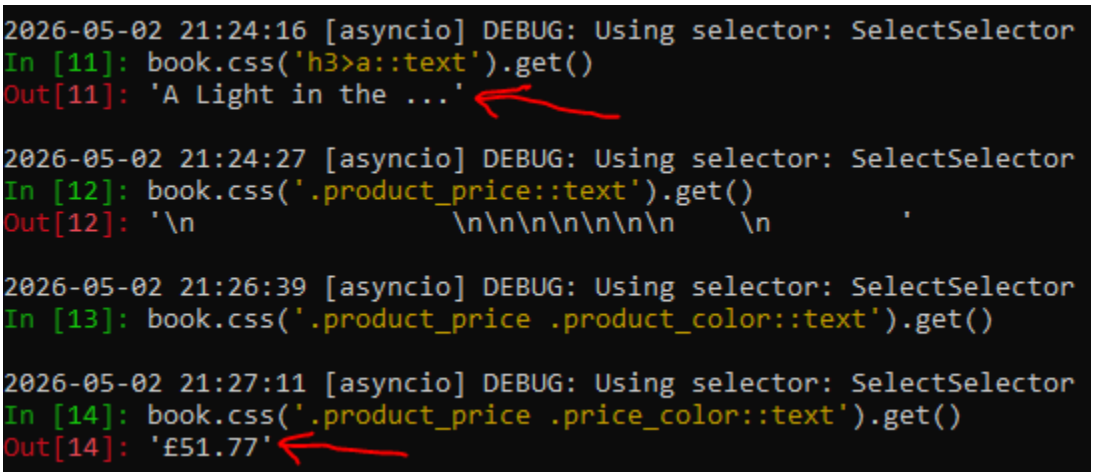
****


2026-05-02 21:16:37 [asyncio] DEBUG: Using selector: SelectSelector
In [4]: response.css('article.product_pod').get()
Out[4]: '<article class="product_pod">\n \n \n \n \n <img src="media/cache/2c/da/2cdad67c44b002e7ead0cc35693c0e8b.jpg" alt="A Light in the Attic" class="thumbnail">\n \n \n \n \n\n \n \n \n \n \n \n \n \n \n \n <h3>A Light in the ...</h3>\n \n\n \n \n \n\n\n\n\n\n\n £51.77 \n \n\n \n \n \n In stock\n \n \n\n \n \n\n\n\n\n\n\n\n\n <form>\n <button type="submit" class="btn btn-primary btn-block" data-loading-text="Adding...">Add to basket</button>\n </form>\n\n\n \n \n \n \n </article>'
2026-05-02 21:18:31 [asyncio] DEBUG: Using selector: SelectSelector
In [5]: books = response.css('article.product_pod')
2026-05-02 21:18:53 [asyncio] DEBUG: Using selector: SelectSelector
In [6]:

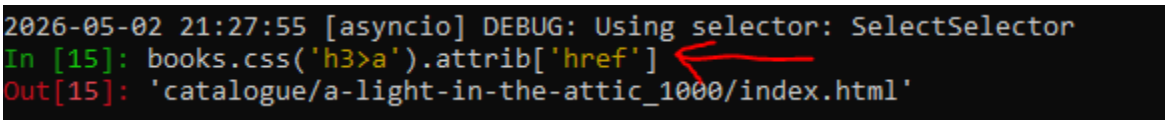
****


2026-05-02 21:18:53 [asyncio] DEBUG: Using selector: SelectSelector
In [6]: len(books)
Out[6]: 20
2026-05-02 21:20:45 [asyncio] DEBUG: Using selector: SelectSelector
In [7]:

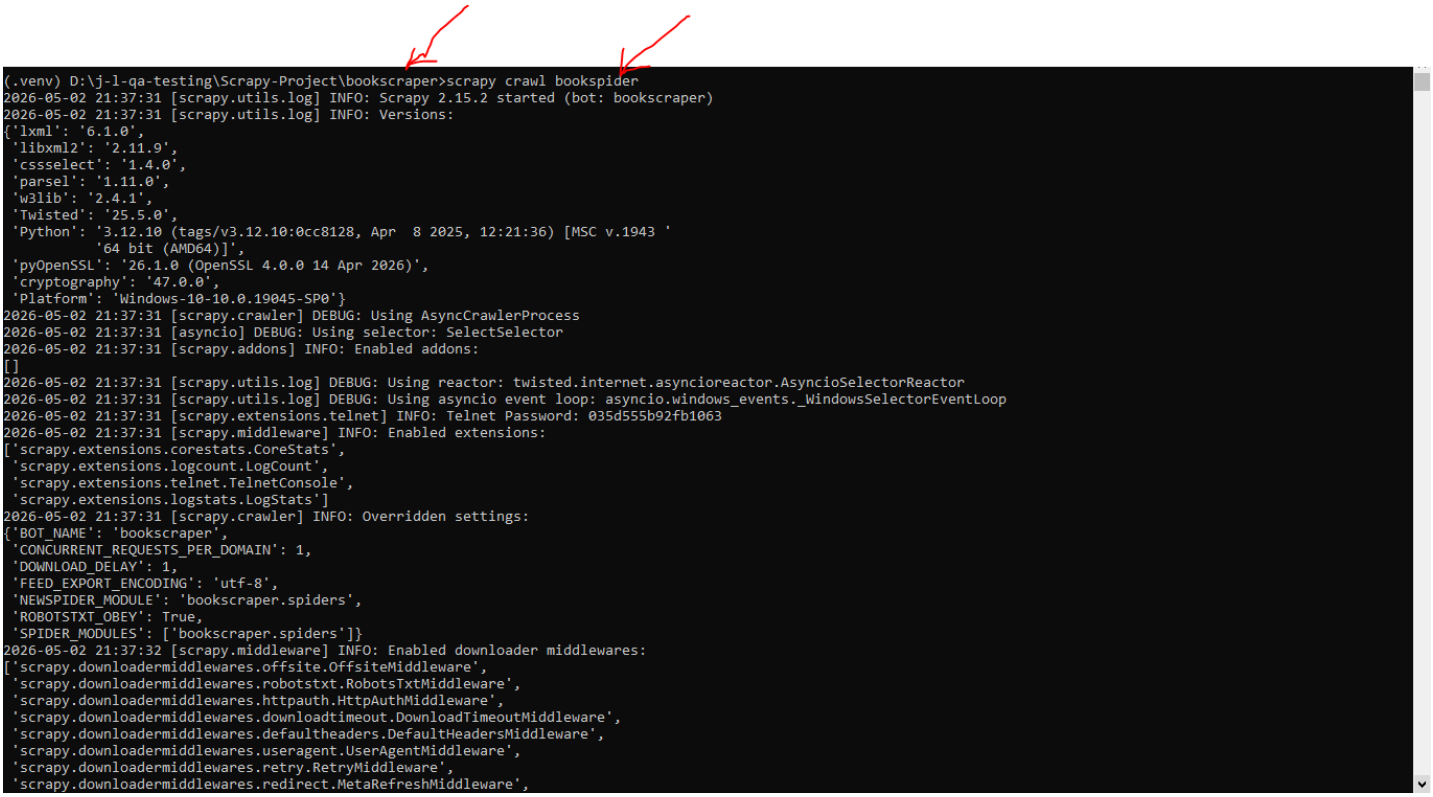
****


2026-05-02 21:24:16 [asyncio] DEBUG: Using selector: SelectSelector
In [11]: book.css('h3>a::text').get()
Out[11]: 'A Light in the ...'
2026-05-02 21:24:27 [asyncio] DEBUG: Using selector: SelectSelector
In [12]: book.css('.product_price::text').get()
Out[12]: '\n \n\n\n\n\n\n\n \n '
2026-05-02 21:26:39 [asyncio] DEBUG: Using selector: SelectSelector
In [13]: book.css('.product_price .product_color::text').get()
2026-05-02 21:27:11 [asyncio] DEBUG: Using selector: SelectSelector
In [14]: book.css('.product_price .price_color::text').get()
Out[14]: '£51.77'

****


2026-05-02 21:27:55 [asyncio] DEBUG: Using selector: SelectSelector
In [15]: books.css('h3>a').attrib['href']
Out[15]: 'catalogue/a-light-in-the-attic_1000/index.html'

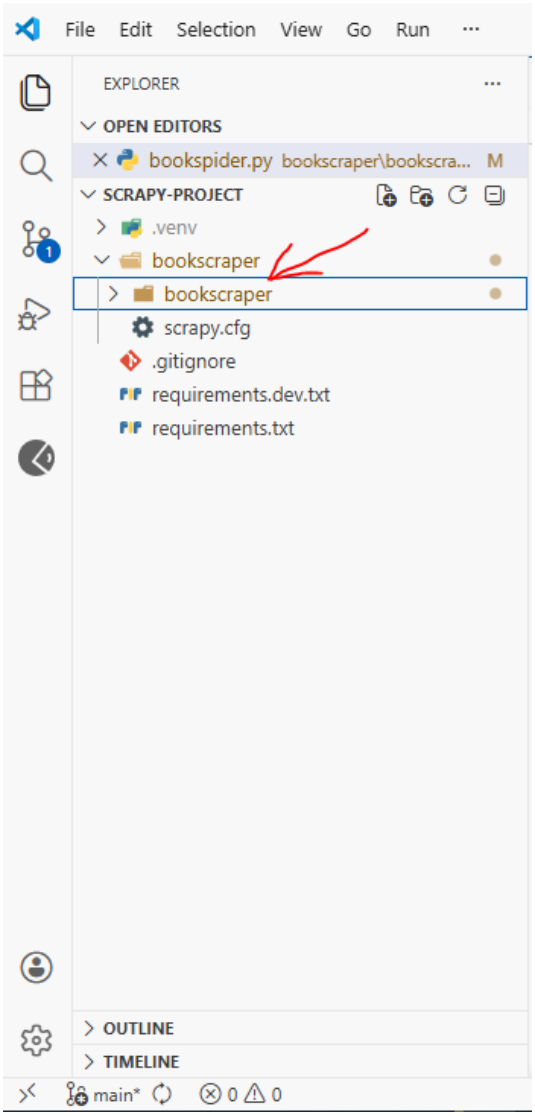
****



```
(.venv) D:\j-l-qa-testing\Scrapy-Project\bookscraper>scrapy crawl bookspider
2026-05-02 21:37:31 [scrapy.utils.log] INFO: Scrapy 2.15.2 started (bot: bookscraper)
2026-05-02 21:37:31 [scrapy.utils.log] INFO: Versions:
{'lxml': '6.1.0',
 'libxml2': '2.11.9',
 'cssselect': '1.4.0',
 'parsel': '1.11.0',
 'w3lib': '2.4.1',
 'Twisted': '25.5.0',
 'Python': '3.12.10 (tags/v3.12.10:0cc8128, Apr  8 2025, 12:21:36) [MSC v.1943 '
           '64 bit (AMD64)]',
 'pyOpenSSL': '26.1.0 (OpenSSL 4.0.0 14 Apr 2026)',
 'cryptography': '47.0.0',
 'Platform': 'Windows-10-10.0.19045-SP0'}
2026-05-02 21:37:31 [scrapy.crawler] DEBUG: Using AsyncCrawlerProcess
2026-05-02 21:37:31 [asyncio] DEBUG: Using selector: SelectSelector
2026-05-02 21:37:31 [scrapy.addons] INFO: Enabled addons:
[]
2026-05-02 21:37:31 [scrapy.utils.log] DEBUG: Using reactor: twisted.internet.asyncioreactor.AsyncioSelectorReactor
2026-05-02 21:37:31 [scrapy.utils.log] DEBUG: Using asyncio event loop: asyncio.windows_events._WindowsSelectorEventLoop
2026-05-02 21:37:31 [scrapy.extensions.telnet] INFO: Telnet Password: 035d555b92fb1063
2026-05-02 21:37:31 [scrapy.middleware] INFO: Enabled extensions:
['scrapy.extensions.corestats.CoreStats',
 'scrapy.extensions.logcount.LogCount',
 'scrapy.extensions.telnet.TelnetConsole',
 'scrapy.extensions.logstats.LogStats']
2026-05-02 21:37:31 [scrapy.crawler] INFO: Overridden settings:
{'BOT_NAME': 'bookscraper',
 'CONCURRENT_REQUESTS_PER_DOMAIN': 1,
 'DOWNLOAD_DELAY': 1,
 'FEED_EXPORT_ENCODING': 'utf-8',
 'NEWSPIDER_MODULE': 'bookscraper.spiders',
 'ROBOTSTXT_OBEY': True,
 'SPIDER_MODULES': ['bookscraper.spiders']}
2026-05-02 21:37:32 [scrapy.middleware] INFO: Enabled downloader middlewares:
['scrapy.downloadermiddlewares.offsite.OffsiteMiddleware',
 'scrapy.downloadermiddlewares.robotstxt.RobotsTxtMiddleware',
 'scrapy.downloadermiddlewares.httpauth.HttpAuthMiddleware',
 'scrapy.downloadermiddlewares.downloadtimeout.DownloadTimeoutMiddleware',
 'scrapy.downloadermiddlewares.defaultheaders.DefaultHeadersMiddleware',
 'scrapy.downloadermiddlewares.useragent.UserAgentMiddleware',
 'scrapy.downloadermiddlewares.retry.RetryMiddleware',
 'scrapy.downloadermiddlewares.redirect.MetaRefreshMiddleware',
```

****

From this directory, we can run the spider, using the command above:



****

The simple spider to crawl the whole site:

```
import scrapy

class BookspiderSpider(scrapy.Spider):
    name = "bookspider"
    allowed_domains = ["books.toscrape.com"]
    start_urls = ["https://books.toscrape.com"]

    def parse(self, response):
        books = response.css('article.product_pod')
        for book in books:
            yield{
                'name': book.css('h3 a::text').get(),
                'price': book.css('.product_price .price_color::text').get(),
                'url': books.css('h3 a').attrib['href']
            }
        next_page = response.css('li.next a::attr(href)').get()

        if next_page is not None:
            if 'catalogue/' in next_page:
                next_page_url = 'https://books.toscrape.com/' + next_page
            else:
                next_page_url = 'https://books.toscrape.com/catalogue/' +
next_page
            yield response.follow(next_page_url, callback=self.parse)

             ***************************************************
```

In the settings.py file for saving the output:

```
FEEDS = {
    "booksdata.json": { 'format': 'json' }
}
```

And for the logging, in the spider:

```
custom_settings = {
        'LOG_FILE': 'result.log',
        'LOG_LEVEL': 'INFO'
    }
             ***************************************************
```

To override the **settings.py** file configurations, we can use **custom_settings** per the spider files.

****

```
import mysql.connector

class SaveToMySQLPipeline:
    def __init__(self):
        self.conn = mysql.connector.connect(
            host = 'localhost',
            user = 'root',
            password = '123',
            database = 'scrapbooks'
        )

        self.cur = self.conn.cursor()
        self.cur.execute("""
            CREATE TABLE IF NOT EXISTS books (
                id INT AUTO_INCREMENT PRIMARY KEY,
                url VARCHAR(1024) UNIQUE NOT NULL,
                title VARCHAR(255),
                product_type VARCHAR(100),
                price_excl_tax DECIMAL(10, 2),
                price_incl_tax DECIMAL(10, 2),
                tax DECIMAL(10, 2),
                availability VARCHAR(100),
                num_reviews INT,
                stars VARCHAR(50),
                category VARCHAR(100),
                description TEXT,
                price DECIMAL(10, 2),
                created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
                updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP ON UPDATE
CURRENT_TIMESTAMP
            ) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci;
""")
    def process_item(self, item, spider):
        self.cur.execute("""
            INSERT INTO books (
                url, title, product_type, price_excl_tax, price_incl_tax, tax,
                availability, num_reviews, stars, category, description, price
            ) VALUES (%s, %s, %s, %s, %s, %s, %s, %s, %s, %s, %s, %s)
            ON DUPLICATE KEY UPDATE
                title = VALUES(title),
                price = VALUES(price),
```

```
                availability = VALUES(availability),
                updated_at = CURRENT_TIMESTAMP;
        """, (
            item.get('url'),
            item.get('title'),
            item.get('product_type'),
            item.get('price_excl_tax'),
            item.get('price_incl_tax'),
            item.get('tax'),
            item.get('availability'),
            item.get('num_reviews'),
            item.get('stars'),
            item.get('category'),
            item.get('description'),
            item.get('price')
        ))

        # Required to actually persist the data in MySQL
        self.conn.commit()

        # Required by Scrapy's pipeline contract
        return item
    def close_spider(self, spider):
        self.cur.close()
        self.conn.close()
            ******************************************************
```

To enable the save **to MySQL pipeline**, in the settings.py file (Only remove the comment):

```
ITEM_PIPELINES = {
   "bookscraper.pipelines.BookscraperPipeline": 300,
#    "bookscraper.pipelines.SaveToMySQLPipeline": 400,
}
```